

Ödev 15: Bir Rehber

Bir arama API'si kurun, `orElse` tuzağını izleyebileceğiniz bir sayaçla kanıtlayın ve dört yanlış kullanımı düzeltin.

Veri

```
record Contact(String name, String email, String phoneOrNull) { }

Map.of(
    "ada", new Contact("ada", "ada@example.com", "555-0100"),
    "grace", new Contact("grace", "grace@example.com", null),
    "alan", new Contact("alan", "alan@example.com", "555-0199"));
```

`grace` kaydının telefonu olmadığına ve alanın `phoneOrNull` olarak adlandırıldığına dikkat edin; sebebini üçüncü bölümde uygulayacaksınız.

Birinci Bölüm: Kutudan Çıkarmadan Arama

`ContactBook.java` dosyasını klasik biçimde oluşturun.

`Optional<Contact>` döndüren `find(String name)` metodu yazın.

Sonra `ada`, `grace` ve `nobody` için her kişinin telefonunu ve e-posta alan adını yazdırın. Eksik değerler okunabilir bir yedek alsın.

Kurallar:

- Bu bölümde hiçbir yerde `isPresent` yok
- Dosyanın tamamında hiçbir yerde `get` yok
- Telefon araması `map` değil `flatMap` gerektirmeli. İkisini de gösterin ve `map` sürümünün neden bir `orElse(String)` içine bile derlenmediğini söyleyin

Sonra `or()` ile üç yedeği zincirleyin ve `orElse` seçeneğinin o zinciri kurmak için neden kullanılamayacağını bir yorumda açıklayın.

Bir ad listesini `find` üzerinden eşleyip yalnızca isabetleri toplayarak bitirin; filtre değil `Optional::stream` kullanın.

İkinci Bölüm: Tuzağı Kanıtlayın

Bir `static int defaultLookups` sayacı ve onu artıran bir `lookupDefaultEmail()` metodu ekleyin.

Sonra üç durumu çalıştırın ve her birinden sonra sayacı yazdırın:

Durum	Beklenen
Dolu bir Optional üzerinde <code>orElse(lookupDefaultEmail())</code>	?
Aynısında <code>orElseGet(ContactBook::lookupDefaultEmail)</code>	?
Boş bir Optional üzerinde <code>orElseGet(...)</code>	?

Çalıştırmadan önce ne tahmin ettiğinizi, sonra gerçekte ne olduğunu yazın.

Açıklayın

Kendi kelimelerinizle bir yorumda:

- Değer mevcutken `orElse` argümanı neden yine de çalışıyor
- `orElseGet` neden çalışmıyor
- Bunun neden yalnızca bir performans notu değil bir doğruluk hatası olduğu.** Cevabınız sayaca atıfta bulunmalı ve gerçek kodda ne olacağını söylemeli.

Üçüncü Bölüm: Dört Yanlış Kullanımı Düzeltin

Her birinin düzeltilmiş biçimini, neyin yanlış olduğunu isimlendiren bir yorumla gösterin:

- `isPresent` sonra `get` — tek bir zincirlenmiş çağrıyla değiştirin
- Alan olarak** `Optional` — `phoneOrNull` alanını düz tutun, erişimciden `Optional` döndürün, ve `Optional` bir alanın neye mal olacağını söyleyin
- Parametre olarak** `Optional` — isteğe bağlı bir etiket alan bir metodu bunun yerine bir **aşırı yükleme** ile yazın
- `Optional<List<T>>` — boş liste döndüren bir arama yazın, ve sarmalanmış sürümün çağırana kaç durum dayatacağını söyleyin

Sonra bir `get()` çağrısını mesajlı `orElseThrow` ile değiştirin. Hem o mesajı hem `get()` metodunun söyleyeceğini yazdırın ve farkı yorumlayın.

Dördüncü Bölüm: Sizi Neye Karşı Korumadığı

`Optional<String>` döndürdüğü bildirilen ama `null` döndüren bir metot yazın.

Çağırın, ne geldiğini yazdırın, sonra sonuç üzerinde herhangi bir metot çağırın ve ne olduğunu yakalayın.

Bir yorumda bunun neden düz null döndüren bir metottan daha kötü olduğunu ve `Optional` sınıfının neden bir garanti değil bir gelenek olduğunu kanıtladığını açıklayın. Derleyicinin işin içinde olduğu Kotlin ya da Rust ile karşılaştırın.

Kabul Kriterleri

- ☐ `java ContactBook.java` ile çalışıyor
- ☐ Dosyada hiçbir yerde `get()` geçmiyor
- ☐ Yanlış kullanımı gösterdiğiniz yer hariç hiçbir yerde `isPresent` geçmiyor
- ☐ Telefon araması `flatMap` kullanıyor, her iki biçim gösteriliyor
- ☐ `or()` üç yedeği zincirliyor, `orElse` neden yapamaz yorumuyla
- ☐ Iskalamaları düşürmek için `Optional::stream` kullanılıyor
- ☐ Sayaç üç farklı sonuçla üç kez yazdırılıyor
- ☐ Açıklama argüman değerlendirmesini ve neden doğruluk hatası olduğunu kapsıyor
- ☐ Dört yanlış kullanımın hepsi düzeltilmiş gösteriliyor, her biri bir yorumla
- ☐ `orElseThrow` mesajı `get()` mesajıyla karşılaştırılıyor
- ☐ Null döndüren `Optional` metodu gösteriliyor ve açıklanıyor

İleri Seviye

Var olan ilk adı döndüren, gereğinden fazla arama değerlendirmeyen `Optional<Contact>` `findAny(String... names)` yazın.

Sonra sürümünüzün tembel olup olmadığını, ve bir stream kullandıysanız `findFirst` metodunun `or()` üzerindeki bir döngünün yapmayacağı neyi yaptığını bir yorumda yazın.

İpucu, birinci bölümün `flatMap` noktası oturmuyorsa

`find("ada").map(Contact::phone).orElse("none")` yazmayı deneyin ve derleyici hatasını okuyun. Tip `Optional<Optional<String>>` ve onun üzerindeki `orElse` bir `String` değil bir `Optional<String>` istiyor. Hatanın kendisi ders.